S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 07

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Name: Sonu Shingare

USN: CD24053

Semester / Year:

Academic Session:

Date of Performance:

Date of Submission:

❖ CODE :

```
GNU nano 8.7                                                    s
include <stdio.h>
include <stdbool.h>

define P 5   // Number of processes
define R 4   // Number of resources

nt main() {
   int allocation[P][R], max[P][R], need[P][R];
   int available[R];
   int finish[P] = {0};
   int safeSeq[P];
   int work[R];

   char *processes[P] = {
       "Google Drive",
       "Firefox",
       "Word Processor",
       "Excel",
       "PowerPoint"
   };

   char *resources[R] = {
       "Printer",
       "ROM",
       "Hard Disk",
       "RAM"
   };

   printf("Enter Allocation Matrix (5x4):\n");
   for(int i = 0; i < P; i++) {
       printf("For %s:\n", processes[i]);
       for(int j = 0; j < R; j++) {
           printf("  %s: ", resources[j]);
           scanf("%d", &allocation[i][j]);
       }
   }

   printf("\nEnter Maximum Matrix (5x4):\n");
   for(int i = 0; i < P; i++) {
       printf("For %s:\n", processes[i]);
       for(int j = 0; j < R; j++) {
           printf("  %s: ", resources[j]);
           scanf("%d", &max[i][j]);
```

```
GNU nano 8.7                                                    safe.c
   }

   // Banker's Algorithm
   int count = 0;

   while(count < P) {
       bool found = false;

       for(int i = 0; i < P; i++) {
           if(!finish[i]) {
               bool possible = true;

               for(int j = 0; j < R; j++) {
                   if(need[i][j] > work[j]) {
                       possible = false;
                       break;
                   }
               }

               if(possible) {
                   for(int j = 0; j < R; j++)
                       work[j] += allocation[i][j];

                   safeSeq[count++] = i;
                   finish[i] = 1;
                   found = true;
               }
           }
       }

       if(!found) {
           printf("\nSystem is NOT in a safe state!\n");
           return 0;
       }
   }

   printf("\nSystem is in a SAFE state.\nSafe sequence:\n");
   for(int i = 0; i < P; i++) {
       printf("%s", processes[safeSeq[i]]);
       if(i != P - 1)
           printf(" -> ");

   return 0;
```

❖ OUTPUT :

```
~
n
sonus@DESKTOP-5EMSD6U MINGW64 ~
$ nano safe.c

sonus@DESKTOP-5EMSD6U MINGW64 ~
$ gcc safe.c -o safe

sonus@DESKTOP-5EMSD6U MINGW64 ~
$ ./safe
Enter Allocation Matrix (5x4):
For Google Drive:
  Printer: 4
  ROM: 3
  Hard Disk: 1
  RAM: 5
For Firefox:
  Printer: 2
  ROM: 1
  Hard Disk: 5
  RAM: 2
For Word Processor:
  Printer: 2
  ROM: 1
  Hard Disk: 4
  RAM: 2
For Excel:
  Printer: 5
  ROM: 3
  Hard Disk: 2
  RAM: 1
For PowerPoint:
  Printer: 2
  ROM: 1
  Hard Disk: 3
  RAM: 5

Enter Maximum Matrix (5x4):
For Google Drive:
  Printer: 2
  ROM: 4
  Hard Disk: 5
  RAM: 1
For Firefox:
  Printer: 2
  ROM: 3
  Hard Disk: 1
  RAM: 4
For Word Processor:
  Printer: 2
```

```
~
  RAM: 5

Enter Maximum Matrix (5x4):
For Google Drive:
  Printer: 2
  ROM: 4
  Hard Disk: 5
  RAM: 1
For Firefox:
  Printer: 2
  ROM: 3
  Hard Disk: 1
  RAM: 4
For Word Processor:
  Printer: 2
  ROM: 4
  Hard Disk: 5
  RAM: 1
For Excel:
  Printer: 2
  ROM: 1
  Hard Disk: 3
  RAM: 4
For PowerPoint:
  Printer: 1
  ROM: 3
  Hard Disk: 1
  RAM: 3

Enter Available Resources:
  Printer: 1
  ROM: 3
  Hard Disk: 1
  RAM: 2

Need Matrix:
-2 1 4 -4
0 2 -4 2
0 3 1 -1
-3 -2 1 3
-1 2 -2 -2

System is in a SAFE state.
Safe sequence:
Firefox ->
```